Before start, make sure you've read the following materials:

- Tour of Go: Interfaces (from "Interfaces" until "Exercise: Errors")
- Tour of Go: Generics (full section)
- Go FAQ: Why is my nil error value not equal to nil?
- Go Blog: An Introduction To Generics
- Go Blog: When To Use Generics
- Deep dive into Go interfaces
- Nil is not always nil
- Go 1.18 generics briefing
- Generics in Go: when you need them and when you don't
- Go by Example: Interfaces, Generics

Part 1: Interface semantics

> **Important**
>
> ***An interface describes behaviour, not data. It is a list of method signatures. Any type that has these methods implements the interface automatically — there is no `implements` keyword.***

Declaring and implementing an interface

```
type Shape interface {
    Area() float64
    Perimeter() float64
}

type Rect struct{ W, H float64 }

func (r Rect) Area() float64      { return r.W * r.H }
func (r Rect) Perimeter() float64 { return 2 * (r.W + r.H) }

type Circle struct{ R float64 }

func (c *Circle) Area() float64      { return math.Pi * c.R * c.R }
func (c *Circle) Perimeter() float64 { return 2 * math.Pi * c.R }

func printInfo(s Shape) {
    fmt.Printf("%-14T area=%6.2f perimeter=%6.2f\n", s, s.Area(), s.Perimeter())
}

func main() {
    shapes := []Shape{Rect{3, 4}, &Circle{1}}
    for _, s := range shapes {
        printInfo(s)
    }
}
```

```
main.Rect      area= 12.00 perimeter= 14.00
*main.Circle   area=  3.14 perimeter=  6.28
```

Key Points:

- `Rect` never says "I implement `Shape`". It just has the right methods. This is called **implicit implementation** (or "structural typing").
- `printInfo` doesn't know which concrete type it gets. The call `s.Area()` is decided **at run time** — this is Go's polymorphism.
- Remember Week 4: `Circle` has **pointer** receivers, so only `*Circle` is a `Shape`, not `Circle`.

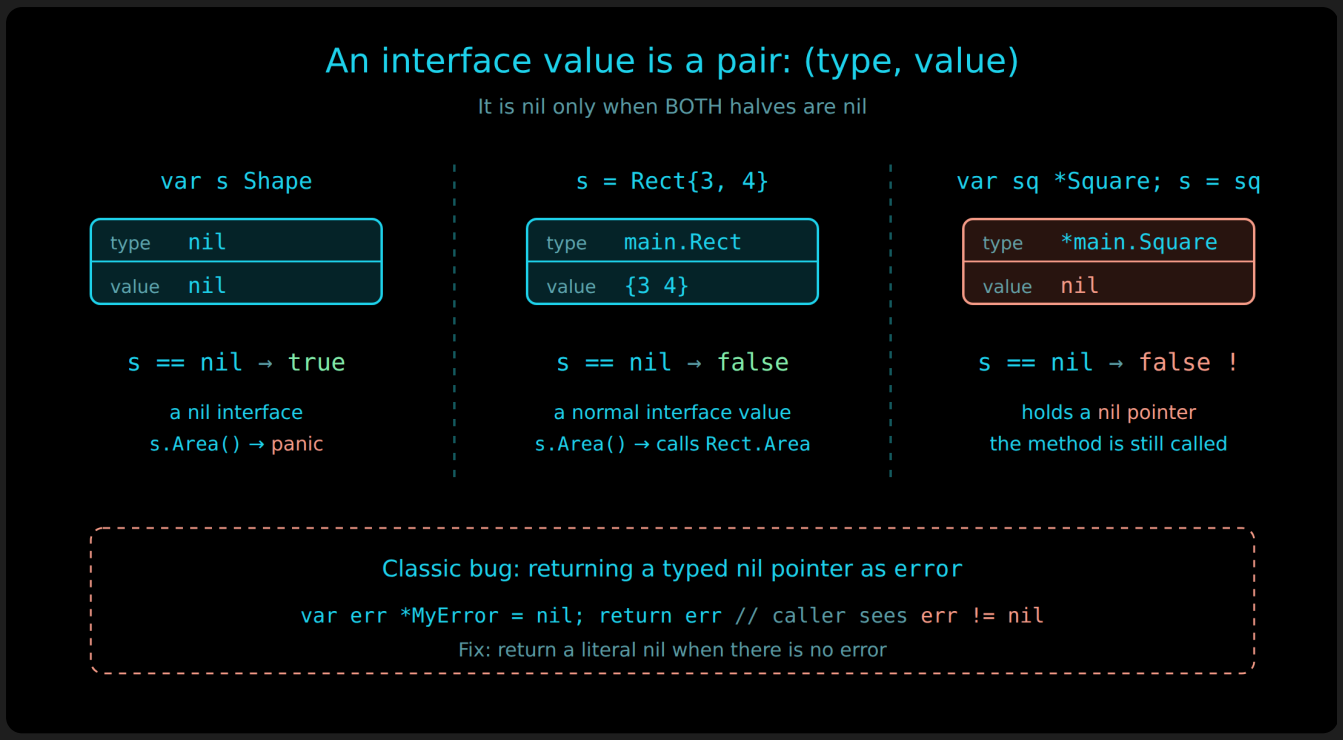
> **Tip**
>
> Ask the compiler to check that a type implements an interface:
>
> ```
> var _ Shape = Rect{}
> var _ Shape = (*Circle)(nil)
> ```
>
> If someone renames a method, the build fails right here — not somewhere far away.

What is inside an interface value?

An interface variable stores **two things**: the **dynamic type** of the value put into it, and the **value** itself.

```
var s Shape
fmt.Printf("%v %T\n", s, s) // ?
s = Rect{1, 2}
fmt.Printf("%v %T\n", s, s) // ?
s = &Circle{2}
fmt.Printf("%v %T\n", s, s) // ?
```

> **Question**
>
> Predict the three lines. What does `%T` print for an interface variable — the interface type `Shape` or something else?



The empty interface: `any`

`interface{}` has no methods, so every type implements it. Since Go 1.18 it has a short alias: `any`.

```
var x any
x = 42
x = "hello"
x = []int{1, 2}
```

> ⚠ **Caution**
>
> **`any` switches off the type checker. You can put anything in, but to do something useful with it you must get the concrete type back out (Part 2). Use `any` only when you really don't know the type — e.g. `fmt.Println(a ...any)`.**

Small interfaces from the standard library

Go interfaces are usually tiny — one or two methods. You already used some of them:

Interface	Method	Who implements it
`fmt.Stringer`	`String() string`	anything you want to print nicely
`error`	`Error() string`	every error value
`io.Writer`	`Write(p []byte) (n int, err error)`	`os.Stdout`, files, `*strings.Builder`, HTTP responses
`io.Reader`	`Read(p []byte) (n int, err error)`	`os.Stdin`, files, network connections

```
func writeReport(w io.Writer, name string, score int) {
    fmt.Fprintf(w, "%s: %d points\n", name, score)
}

func main() {
    writeReport(os.Stdout, "Aruzhan", 95) // print to the terminal

    var sb strings.Builder // remember Week 3
    writeReport(&sb, "Dias", 80)          // write into a string
    fmt.Print(sb.String())
}
```

One function, two very different destinations. That is the power of a small interface.

Embedding interfaces

Just like structs (Week 4), interfaces can be built from smaller ones:

```
type Logger interface{ Log(msg string) }
type Saver  interface{ Save(data string) error }

type LogSaver interface {
    Logger
    Saver
}
```

`io.ReadWriter` **in the standard library is built exactly this way from** `io.Reader` **and** `io.Writer`**.**

> **Task**
>
> ### Task 1: Notifications
>
> **1. Declare** `type Notifier interface { Notify(msg string) error }`**.**
>
> **2. Implement it with three types:** `Email{Address}`**,** `SMS{Phone}` **and** `Telegram{Chat}`**.** `SMS` **must return an error if the message is longer than 20 characters.** `Telegram` **must count how many messages it has sent (think about the receiver type!).**
>
> **3. Write** `Broadcast(ns []Notifier, msg string) []error` **that sends the message through every notifier and collects the errors.**
>
> **4. Add compile-time checks (** `var _ Notifier = ...` **) for all three types.**
>
> **Participation Task: write a function that accepts an** `io.Writer` **and prints a table of your group's grades. Call it once with** `os.Stdout` **and once with a** `*strings.Builder`**.**

Part 2: Type assertions and type switches

Type assertion: "I believe this interface holds a `T`"

```go
var i any = "Go"

s := i.(string)       // OK: s is a string
fmt.Println(s)

n, ok := i.(int)      // the "comma ok" form never panics
fmt.Println(n, ok)    // 0 false

f := i.(float64)      // ???
```

> ⚠ **Caution**
>
> **The single-value form panics if you are wrong:**

```
panic: interface conversion: interface {} is string, not float64
```

> **Tip**
>
> **Use `v, ok := i.(T)` whenever you are not 100% sure. It's the same "comma ok" idiom you used with maps in Week 3.**

Asserting to another interface

`T` in `i.(T)` can also be an interface. Then the question is "does the value inside have these methods?"

```go
type Celsius float64

func (c Celsius) String() string { return fmt.Sprintf("%.1f°C", float64(c)) }

var t any = Celsius(36.6)
if st, ok := t.(fmt.Stringer); ok {
    fmt.Println("Stringer:", st.String()) // Stringer: 36.6°C
}
```

This is how `fmt.Println` decides whether to call your `String()` method.

Type switch

When there are several possible types, use a type switch instead of a chain of assertions:

```
func describe(v any) string {
    switch x := v.(type) {
    case nil:
        return "nil"
    case int, int64:
        return fmt.Sprintf("integer %v", x)
    case string:
        return fmt.Sprintf("string of %d bytes", len(x)) // x is a string here
    case []int:
        return fmt.Sprintf("slice with %d ints", len(x)) // x is a []int here
    case error:
        return "error: " + x.Error()
    case fmt.Stringer:
        return "Stringer: " + x.String()
    default:
        return fmt.Sprintf("unknown type %T", x)
    }
}
```

Key Points:

- `v.(type)` is allowed only inside a `switch`.
- In a case with one type, `x` has that type. In a case with several types (`case int, int64:`) or in `default`, `x` keeps the original type `any`.
- Cases are checked top to bottom; the first match wins.

> ⓘ **Question**
>
> **Why does this code not compile?**

```
switch x := v.(type) {
case int, int64:
    return x + 1
}
```

Question

`Timeout` has both `Error()` and `String()` methods. What will this program print? Would the answer change if you swap the first two cases?

```
type Timeout struct{ Sec int }

func (t Timeout) Error() string  { return fmt.Sprintf("timeout after %ds", t.Sec) }
func (t Timeout) String() string { return "Timeout{" + fmt.Sprint(t.Sec) + "}" }

func kind(v any) string {
    switch v.(type) {
    case fmt.Stringer:
        return "Stringer"
    case error:
        return "error"
    case Timeout:
        return "Timeout"
    }
    return "other"
}

func main() {
    fmt.Println(kind(Timeout{5}))
    var e error = Timeout{3}
    fmt.Println(e) // Error() or String()?
}
```

Task

Task 2: Describe anything

1. Copy `describe` from above and call it with: `nil`, `7`, `int64(7)`, `"привет"`, `[]int{1, 2, 3}`, `errors.New("disk full")`, `Celsius(21.5)`, `3.14`, `struct{}{}`.

2. Before running, write down what you expect for each value. Explain every mismatch. (Hint: why is `"привет"` 12 bytes? — Week 3.)

3. Add a case for `bool` and for `map[string]int`.

Participation Task: rewrite `describe` without a type switch, using only `v, ok := i.(T)` assertions. Which version is easier to read?

Note

For errors, prefer `errors.As` and `errors.Is` over raw assertions: they also look inside wrapped errors (`fmt.Errorf("...: %w", err)`). You will use them in the error-handling week.

Part 3: Nil interfaces

Important

An interface value is `nil` only when both its type and its value are `nil`.
An interface that holds a `nil` pointer is not `nil`.

```go
type Shape interface{ Area() float64 }

type Square struct{ Side float64 }

func (s *Square) Area() float64 {
    if s == nil {
        return 0
    }
    return s.Side * s.Side
}

func main() {
    var s Shape            // nil interface
    fmt.Println(s == nil)  // ?

    var sq *Square         // nil pointer
    s = sq                 // interface holds (*Square, nil)
    fmt.Println(s == nil)  // ?
    fmt.Println(s.Area())  // ?

    s = nil
    fmt.Println(s.Area())  // ?
}
```

Question

Predict all four lines. Why can Go call `Area()` when the interface holds a `nil` pointer, but not when the interface itself is `nil`?

Look at the diagram above (Part 1) again: the third column is exactly this situation.

The classic bug: a nil error that is not nil

```go
type ValidationError struct {
    Field string
}

func (e *ValidationError) Error() string { return "invalid field: " + e.Field }

func validate(age int) error {
    var err *ValidationError
    if age < 0 {
        err = &ValidationError{Field: "age"}
    }
    return err
}

func main() {
    err := validate(20)
    if err != nil {
        fmt.Println("error:", err)
    }
}
```

> ⚠ **Caution**
>
> **The age is valid, yet this program prints an error. The function returns an `error` interface that holds a `(*ValidationError, nil)` pair — and that pair is not `nil`.**

> **Tip**
>
> **Rules that prevent this bug:**
>
> **1. Functions should return the `error` interface, not a concrete type like `*ValidationError`.**
>
> **2. When there is no error, `return nil` — literally.**
>
> **3. Don't keep a typed pointer variable "for later" and return it.**

Task

Task 3: Find the bug

1. Run the `validate` program above. What exactly does it print? Why is the value shown as `<nil>` even though the `if` was entered? (Hint: `fmt` protects you when a method panics on a nil receiver.)

2. Print `err == nil` and `%T` of `err` to prove your explanation.

3. Fix `validate` following the rules above. Then call it with `-1` and use `errors.As` to get the `Field` out of the error.

Task

Task 4: Solve the problem from the tour of Go -> Exercise: Errors

On the defence, explain why `fmt.Sprint(e)` inside the `Error()` method would send the program into an infinite loop, and how `go vet` detects it.

Part 4: Generics and constraints

The problem

Before Go 1.18 you had to write the same function for every type, or use `any` and lose type safety:

```
func SumInts(nums []int) int {
    var s int
    for _, n := range nums {
        s += n
    }
    return s
}

func SumFloats(nums []float64) float64 {
    var s float64
    for _, n := range nums {
        s += n
    }
    return s
}
```

The solution: type parameters

```
type Number interface {
    ~int | ~int64 | ~float64
}

func Sum[T Number](nums []T) T {
    var s T // the zero value of T
    for _, n := range nums {
        s += n
    }
    return s
}

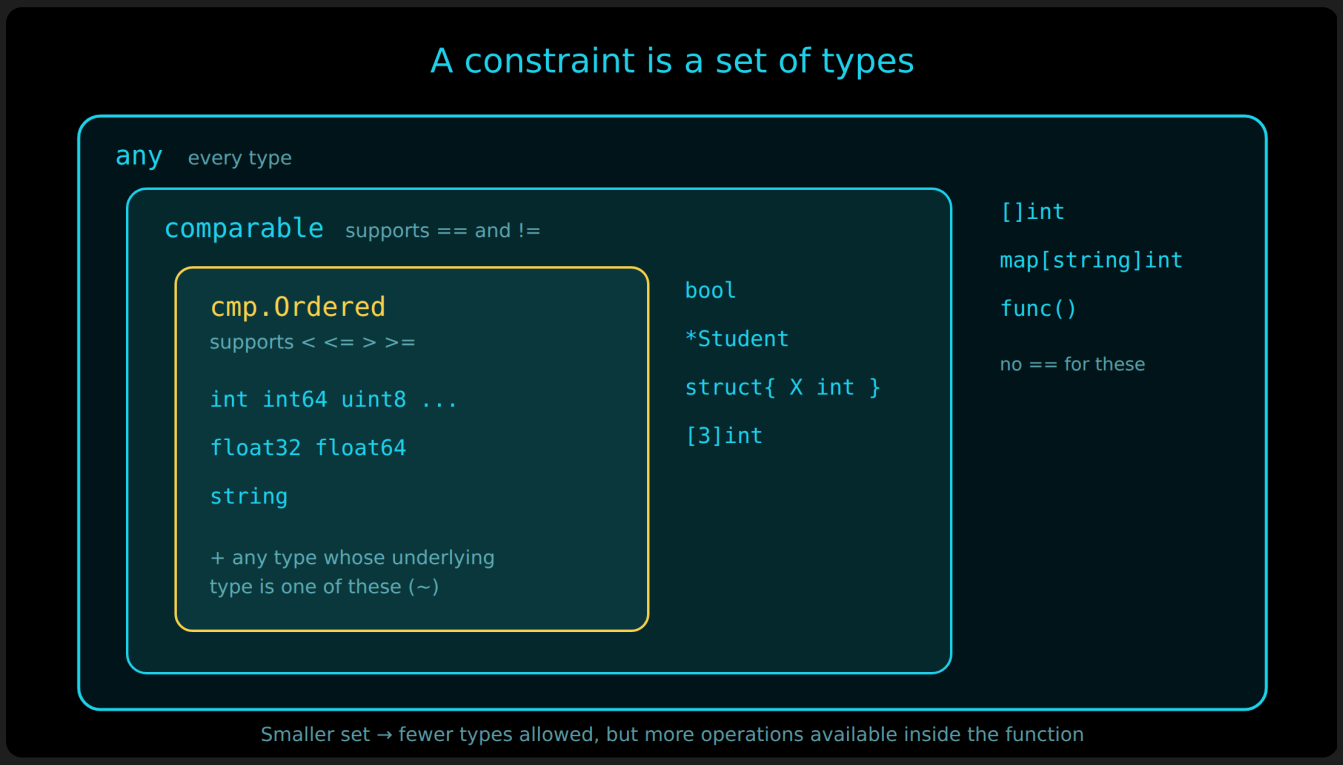
func main() {
    fmt.Println(Sum([]int{1, 2, 3}))        // 6    — T inferred as int
    fmt.Println(Sum([]float64{1.5, 2.5}))   // 4    — T inferred as float64
    fmt.Println(Sum[int64]([]int64{10, 20})) // 30  — T written explicitly
}
```

Key Points:

- **`[T Number]` is the type parameter list: `T` is the name, `Number` is the constraint.**
- **Usually the compiler infers `T` from the arguments. You can also write it explicitly: `Sum[int64](...)`.**
- **`var s T` gives you the zero value of whatever `T` is (`0`, `""`, `nil`, ...).**
- **The compiler checks every call: `Sum([]string{"a"})` does not compile.**

Constraints are interfaces

> **Important**
>
> ***A constraint is an interface. But instead of thinking "a list of methods", think "a set of types". The constraint says which types may be used for `T` and which operations are allowed on `T` inside the function.***



Constraint	Allowed types	What you can do with `T`
`any`	every type	assign, pass, return — that's all
`comparable`	types that support `==` (no slices, maps, funcs)	`==`, `!=`, use as a map key
`cmp.Ordered` (Go 1.21)	integers, floats, strings (and `~` of them)	`<`, `<=`, `>`, `>=`, `==`
`int | float64`	exactly `int` and `float64`	operations that **all** of them support (`+`, `<`, ...)
`~int | ~float64`	`int`, `float64` **and** every type built on them	the same operations
`fmt.Stringer`	types with a `String() string` method	call `.String()`

```go
func Max[T cmp.Ordered](a, b T) T {
    if a > b {
        return a
    }
    return b
}

func Contains[T comparable](items []T, target T) bool {
    for _, v := range items {
        if v == target {
            return true
        }
    }
    return false
}

fmt.Println(Max(3, 7), Max("apple", "banana")) // 7 banana
fmt.Println(Contains([]string{"go", "rust"}, "go")) // true
```

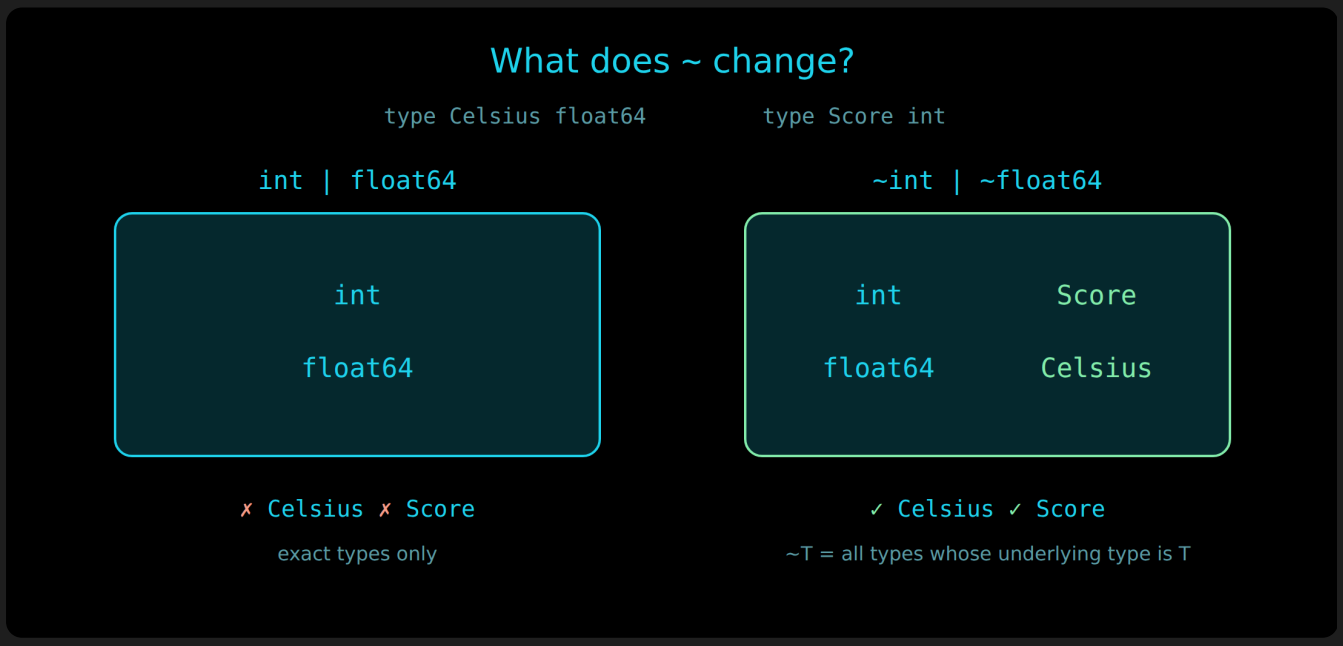
The tilde `~`

```go
type Celsius float64

type Number interface {
    int | float64 // no tilde!
}

Sum([]Celsius{20.5, 21.5})
```

```
Celsius does not satisfy Number (possibly missing ~ for float64 in Number)
```



`~float64` means "`float64` and every type whose underlying type is `float64`". Since programs are full of types like `Celsius`, `Score`, `UserID`, you almost always want the `~`.

Constraints with methods

```
func JoinAll[T fmt.Stringer](items []T, sep string) string {
    parts := make([]string, 0, len(items))
    for _, it := range items {
        parts = append(parts, it.String())
    }
    return strings.Join(parts, sep)
}

fmt.Println(JoinAll([]Celsius{20.5, 22, 19.8}, ", ")) // 20.5°C, 22.0°C, 19.8°C
```

Question

Each of these programs fails to compile. Explain why using the type-set idea, and fix each one.

```
// 1
func Contains[T any](items []T, target T) bool {
    for _, v := range items {
        if v == target { return true }
    }
    return false
}

// 2
func Max[T comparable](a, b T) T {
    if a > b { return a }
    return b
}

// 3
type Number interface{ ~int | ~float64 }
var n Number = 5

// 4
func Zero[T any]() T { var z T; return z }
x := Zero()
```

Generic types

Types can have type parameters too. A classic example is a container:

```go
type Stack[T any] struct {
    items []T
}

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }

func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

func main() {
    var s Stack[string]
    s.Push("a")
    s.Push("b")
    fmt.Println(s.Pop()) // b true

    nums := &Stack[int]{}
    nums.Push(10)
    fmt.Println(nums.Pop()) // 10 true
}
```

> **Note**
>
> **Methods use the type's parameters (`Stack[T]`), but a method cannot declare new type parameters of its own:**
>
> ```go
> func (b Box[T]) Map[U any](f func(T) U) Box[U] // syntax error: method must have no type parameters
> ```
>
> **Write a plain generic function instead: `func Map[T, U any](items []T, f func(T) U) []U`.**

Generics in the standard library

You don't have to write everything yourself:

```go
slices.Contains([]string{"go", "rust"}, "rust") // true
slices.Max([]int{4, 9, 2})                      // 9
slices.Sort(nums)
maps.Keys(m)                                    // an iterator over the keys
max(3, 8, 1)                                    // 8 — built-in since Go 1.21
```

Interfaces or generics?

Use an interface when…	Use generics when…
the types have different **behaviour** (`Rect.Area` ≠ `Circle.Area`)	the code is **the same** for every type, only the type differs
you need a mixed collection (`[]Shape` with rects and circles)	you write containers: stacks, queues, sets, trees
you only call methods	you need operators (`+`, `<`, `==`) on the values
e.g. `io.Writer`, `error`, `fmt.Stringer`	e.g. `slices.Contains`, `Sum`, `Stack[T]`

Tip

Rule from the Go blog: write the code with concrete types first. If you catch yourself copying it for another type, then consider a type parameter.

Task

Task 5: Generic toolbox

Write the following functions. Each one must work with `type Score int` too.

1. **`Sum[T Number](nums []T) T` and `Average[T Number](nums []T) (float64, bool)` — `false` for an empty slice.**
2. **`MaxOf[T cmp.Ordered](items []T) (T, bool)` — test it with `int`, `string` and an empty slice.**
3. **`IndexOf[T comparable](items []T, target T) int` — `-1` if not found.**
4. **`Filter[T any](items []T, keep func(T) bool) []T` — keep the scores `>= 75`.**
5. **Remove the `~` from `Number` and read the compiler error. Put it back.**

Task

Task 6: Benchmark interfaces vs generics

In Week 3 you counted allocations, in Week 4 you measured copying. Now compare three ways to sum 1000 numbers:

```go
func SumInts(nums []int) int { /* concrete */ }

func Sum[T Number](nums []T) T { /* generic */ }

func SumAny(nums []any) int {
    total := 0
    for _, n := range nums {
        total += n.(int) // type assertion on every element
    }
    return total
}

func ToAny(nums []int) []any { /* copy every int into a []any */ }
```

Requirements:

- **Write `BenchmarkSumInts`, `BenchmarkSumGeneric`, `BenchmarkSumAny` (convert to `[]any` once, before `b.ResetTimer()`) and `BenchmarkSumAnyWithBoxing` (call `SumAny(ToAny(nums))` inside the loop).**
- **Run them with `go test -bench=. -benchmem` and fill a table with `ns/op`, `B/op`, `allocs/op`.**
- **Explain: why is the generic version as fast as the concrete one? Why is `SumAny` slower even with 0 allocations?**
- **Explain where the `allocs/op` in the last benchmark come from. Why is it not exactly 1001? (Hint: the first number in the slice is `0`.)**

Task

Task 7: Solve the problem from the tour of Go -> Generic types

Add `Push`, `Len` and `String` methods to the `List[T]` type. `Push` must work on an empty (`nil`) list.

⭐ Bonus question (for those who want more)

```go
var a, b any = 1, 1
fmt.Println(a == b)

var c, d any = 1, int64(1)
fmt.Println(c == d)

var e, f any = []int{1}, []int{1}
fmt.Println(e == f)
```

What is printed? One line does something much worse than print `false`. Why does this compile at all?

Theoretical questions for the defence (in-person with Bakytzhan-agai):

- **What is an interface? How does a type implement it in Go? How is this different from Java?**
- **What two things does an interface value contain?**
- **What is the difference between `v := i.(T)` and `v, ok := i.(T)`?**
- **In a type switch, what is the type of `x` in `case int, string:`? Why?**
- **When is an interface value `nil`? Why can a function return a non-nil `error` that holds a `nil` pointer?**
- **What is a type parameter? What is a constraint?**
- **What is the difference between `any`, `comparable` and `cmp.Ordered`?**
- **What does `~` mean in a constraint?**
- **When would you choose an interface over generics, and vice versa?**

Key takeaways:

- An interface is a set of methods. Types implement it implicitly, just by having the methods.
- An interface value is a pair (dynamic type, value). `%T` shows the dynamic type.
- Keep interfaces small: `io.Writer`, `error`, `fmt.Stringer` have one method each.
- `i.(T)` panics on a wrong guess, `v, ok := i.(T)` doesn't. Use a type switch for many types.
- An interface is `nil` only if both its type and value are `nil`. Return a literal `nil` for "no error".
- Generics remove copy-paste: `func Sum[T Number](nums []T) T`.
- A constraint is a set of types. It also decides which operators you may use on `T`.
- Use `~T` to include named types like `type Celsius float64`.
- Different behaviour → interface. Same code, different types → generics.

Defence will be conducted in the next practice lesson, according to your schedule. In-person with your laptops.
